

WTB API - Plan de Desarrollo por Semanas



SEMANA 1: Setup Básico y Configuración Inicial

Día 1-2: Configuración del Proyecto en VS Code

- ☐ Crear nuevo proyecto ASP.NET Core Web API
- ☐ Configurar estructura de carpetas
- ☐ Instalar paquetes NuGet necesarios
- ☐ Configurar appsettings.json con connection string
- ☐ Setup de Entity Framework Core
- ☐ Configurar CORS básico

Día 3-4: Modelos de Base de Datos (Models)

- ☐ Crear modelos de entidad (Entity Models):
 - States.cs
 - Roles.cs
 - DocumentType.cs
 - InternUsers.cs
 - EmployeeInfo.cs
 - Projects.cs
 - Assistance.cs
 - ProjectsAssigns.cs
 - AuditRegister.cs
 - FingerPrint.cs
 - ProjectMaintenance.cs
 - ProjectWarranty.cs

Día 5-7: DbContext y Configuración

- ☐ Crear WTBDbContext
- ☐ Configurar relaciones (Fluent API)
- ☐ Crear migraciones iniciales
- ☐ Probar conexión a base de datos



SEMANA 2: DTOs, AutoMapper y Servicios Base

Día 1-3: DTOs (Data Transfer Objects)

- ☐ Crear DTOs para Request/Response:
 - CreateEmployeeDto, UpdateEmployeeDto, EmployeeResponseDto
 - CreateProjectDto, UpdateProjectDto, ProjectResponseDto
 - CreateAssistanceDto, AssistanceResponseDto
 - LoginDto, UserResponseDto
 - StateDto, RoleDto, DocumentTypeDto

Día 4-5: AutoMapper Configuration

- ☐ Instalar AutoMapper
- ☐ Crear perfiles de mapeo (MappingProfiles)
- ☐ Configurar mapeo bidireccional

Día 6-7: Servicios Base (Services/Repositories)

- ☐ Crear interfaces de repositorio genérico
 - ☐ Implementar Repository Pattern
 - ☐ Crear UnitOfWork pattern
 - ☐ Servicios base para CRUD operations
-



SEMANA 3: Autenticación, Autorización y Middleware

Día 1-3: Sistema de Autenticación

- ☐ Configurar JWT Authentication
- ☐ Crear AuthService para login/register
- ☐ Implementar password hashing (BCrypt)
- ☐ Crear AuthController

Día 4-5: Middleware Personalizado

- ☐ Middleware de manejo de errores globales
- ☐ Middleware de logging
- ☐ Middleware de validación de JWT
- ☐ Rate limiting middleware

Día 6-7: Autorización y Roles

- ☐ Configurar políticas de autorización
- ☐ Implementar role-based authorization

- ☐ Crear atributos personalizados de autorización
-

SEMANA 4: Controllers Principales - Empleados y Usuarios

Día 1-3: Employee Management

- ☐ EmployeeController completo:
 - GET /api/employees (con filtros y paginación)
 - GET /api/employees/{id}
 - POST /api/employees
 - PUT /api/employees/{id}
 - DELETE /api/employees/{id}
 - GET /api/employees/search

Día 4-5: User Management

- ☐ UserController (InternUsers):
 - CRUD operations para usuarios internos
 - Cambio de contraseñas
 - Gestión de roles

Día 6-7: Validaciones y Filtros

- ☐ Implementar FluentValidation
 - ☐ Crear validadores para cada DTO
 - ☐ Filtros de acción personalizados
-

SEMANA 5: Projects y Assignments

Día 1-3: Project Management

- ☐ ProjectController:
 - CRUD completo de proyectos
 - Asignación de empleados a proyectos
 - Estados de proyectos
 - Búsqueda y filtrado

Día 4-5: Project Assignments

- ☐ ProjectAssignmentController:

- Asignar/desasignar empleados
- Historial de asignaciones
- Empleados disponibles por proyecto

Día 6-7: Maintenance y Warranty

- ☐ ProjectMaintenanceController
 - ☐ ProjectWarrantyController
 - ☐ Gestión de costos y estados
-



SEMANA 6: Sistema de Asistencia (Core Business Logic)

Día 1-3: Assistance Management

- ☐ AssistanceController:
 - Check-in/Check-out endpoints
 - Cálculo automático de horas
 - Validaciones de horarios
 - Prevenir doble check-in

Día 4-5: Reportes de Asistencia

- ☐ Reportes por empleado
- ☐ Reportes por proyecto
- ☐ Reportes por rango de fechas
- ☐ Exportación a Excel/PDF

Día 6-7: Business Rules

- ☐ Reglas de negocio para horarios
 - ☐ Validaciones de solapamiento
 - ☐ Cálculos de overtime
-



SEMANA 7: Reportes y Analytics

Día 1-3: Reporting System

- ☐ ReportController:
 - Horas trabajadas por empleado
 - Productividad por proyecto

- Costos por proyecto
- Análisis de asistencia

Día 4-5: Dashboard Data

- ☐ DashboardController:
 - Métricas generales
 - Gráficos de asistencia
 - KPIs principales
 - Datos en tiempo real

Día 6-7: Export Features

- ☐ Exportación de reportes
 - ☐ Generación de PDFs
 - ☐ Archivos Excel con formato
-

SEMANA 8: Testing, Performance y Deploy

Día 1-3: Unit Testing

- ☐ Tests para servicios
- ☐ Tests para controllers
- ☐ Tests de integración
- ☐ Mocking de dependencias

Día 4-5: Performance y Optimización

- ☐ Caching (Memory Cache/Redis)
- ☐ Optimización de queries
- ☐ Paginación eficiente
- ☐ Compression responses

Día 6-7: Documentación y Deploy

- ☐ Configurar Swagger/OpenAPI
 - ☐ Documentar endpoints
 - ☐ Configurar para producción
 - ☐ Deploy inicial
-

Estructura de Carpetas Recomendada

WTB.API/

- |— Controllers/
- |— Models/
- |... |— Entities/
- |... |— DTOs/
- |— Services/
- |... |— Interfaces/
- |... |— Implementations/
- |— Data/
- |... |— Context/
- |... |— Repositories/
- |... |— Configurations/
- |— Middleware/
- |— Helpers/
- |— Mappings/
- |— Validators/
- |— Extensions/

Tecnologías y Paquetes Principales

- **ASP.NET Core 8.0** - Framework principal
- **Entity Framework Core** - ORM
- **AutoMapper** - Object mapping
- **FluentValidation** - Validaciones
- **JWT Bearer** - Autenticación
- **Swagger/OpenAPI** - Documentación
- **Serilog** - Logging
- **BCrypt** - Password hashing
- **AutoMapper** - Object mapping

Notas Importantes

1. **Cada día incluye tiempo para testing** de las funcionalidades implementadas
2. **Commits frecuentes** - Al menos uno por día con funcionalidad completa
3. **Documentación continua** - Cada endpoint debe estar documentado
4. **Code Review** - Revisar código antes de pasar a la siguiente fase
5. **Backup de base de datos** - Hacer respaldos regulares durante desarrollo

